TodoAppWasm Auth and JWT Guide
Junior developer reference for tasks 034_backend_auth_user_accounts and 035_backend_jwt_authorization.

1. Big Picture
Task 034 teaches the backend how to register and login users safely.
Task 035 adds JWT tokens so the backend can protect Todo endpoints and know which user is making each request.

Before authentication, the client could send ownerId in JSON and the backend trusted it.
After JWT authorization, the backend reads the real logged-in user id from the token.

2. Task 034 - User Accounts and Password Hashing
Goal: users register with a password, but the password is never stored as plain text.

User model change:
```
public string PasswordHash { get; set; } = string.Empty;
```

Why: The database stores PasswordHash instead of Password. If someone opens the database, they should not see real passw

UserCreationDto change:
```
public string UserName { get; set; }
public string Password { get; set; }
```

Why: Registration needs username and password from Swagger or the frontend.

Password hashing service:
```
Pbkdf2PasswordHasher.Hash(password)
```

What it does:
- creates a random salt
- derives a secure key from the password and salt
- stores algorithm, iterations, salt, and key together

Important: Hashing is one-way. We do not decrypt passwords. On login, we hash the entered password again and compare has

Password verification:
```
passwordHasher.Verify(dto.Password, existing.PasswordHash)
```

Why FixedTimeEquals is used: it avoids leaking tiny timing clues about the hash comparison.

UserLogic registration:
```
UserName = usertoCreate.UserName
PasswordHash = passwordHasher.Hash(usertoCreate.Password)
Role = UserRoles.User
```

Why Role is set automatically: normal registration should never allow a user to choose Admin in JSON.

Login logic:
  if user does not exist OR password does not match:
      throw Invalid username or password

Why the message is general: attackers should not know whether the username or password was wrong.

Database migration for task 034:
  AddUserPasswordHash adds PasswordHash to Users table.

Command:
  dotnet ef database update --project EfcDataAccess --startup-project WebAPI

If migration is not applied, POST /Users can return 500 because the database is missing PasswordHash.

3. Task 035 - JWT Authorization
Goal: after login, the user receives a token. Todo endpoints require that token.

Login response DTO:
  Token
  User
  ExpiresAtUtc

Why: The frontend or Swagger needs the token for later protected requests.

JWT claims added:
  ClaimTypes.NameIdentifier = user.Id
  ClaimTypes.Name = user.UserName
  ClaimTypes.Role = user.Role

Meaning:
  NameIdentifier tells the backend the logged-in user id.
  Name tells the backend the username.
  Role tells the backend whether the user is User or Admin.

JWT validation in Program.cs checks:
  - issuer
  - audience
  - lifetime
  - signing key

Middleware order:
  app.UseAuthentication();

app.UseAuthorization();

Why order matters:
UseAuthentication reads the token and creates User.Claims.
UseAuthorization checks if the endpoint can run.

Swagger Bearer setup:
Swagger needs AddSecurityDefinition and AddSecurityRequirement to show the Authorize button.

How to paste token in Swagger:
Bearer eyJhbGciOiJIUzI1NiIs...

4. Roles vs Ownership
Roles are account types. Examples: User, Admin.
Ownership is about one specific todo. Example: todo.OwnerId == loggedInUserId.

Clean rule:
Admin and User are roles.
Owner is not a role.
Owner is checked by comparing Todo.OwnerId with the user id from the JWT token.

5. Protected Todo Endpoints
TodosController has [Authorize], so users must login before calling Todo endpoints.

Create Todo:
int userId = GetCurrentUserId();
TodoCreationDto secureDto = new(userId, dto.Title, dto.Description);

Why: The API no longer trusts ownerId from JSON. It uses the user id from the JWT token.

List Todos:
Admin can inspect broader data.
Normal users only see their own todos.

Access check:
If Admin OR todo.OwnerId equals current user id, allow access.
Otherwise return 404.

Why 404 instead of 403: it avoids revealing that another user’s todo exists.

6. How to Test in Swagger
1. Apply migrations so Users table has PasswordHash and Role.
2. Create a user with POST /Users.
3. Login with POST /Users/login.

4. Copy the token from the login response.
5. Click Authorize in Swagger.
6. Paste Bearer {token}.
7. Call Todo endpoints.

7. Common Errors
500 when creating user: database is missing PasswordHash or Role. Run dotnet ef database update.
No Authorize button: Swagger Bearer setup is missing or WebAPI was not restarted.
401 Unauthorized: missing, expired, or invalid token. Login again and paste Bearer token.
404 for existing todo: the todo may belong to another user. Use owner token or Admin token.

8. Final Mental Model
Register: username + password -> hash password -> save UserName, PasswordHash, Role
Login: username + password -> verify hash -> return JWT
Todo request: frontend sends JWT -> backend validates token -> backend reads user id and role -> allows or blocks action